

THE COMPLETE SQL COURSEBOOK

For Data Engineers, Analysts & Data Scientists

A Practical, Project-Based Guide Built Around a Real E-Commerce Business

Dialect: PostgreSQL | Levels: Basics → Intermediate → Advanced
Twelve Modules ▪ Theory + Hands-On Practice ▪ Capstone Projects

NorthStar Commerce

The running case-study business used throughout this book

Table of Contents

Table of Contents	2
Course Overview	5
How to Use This Book	5
Course Map	5
The Case Study: NorthStar Commerce	6
Entity-Relationship Summary	6
categories	6
products	6
customers	6
orders	7
order_items	7
payments	7
reviews	7
Module 1: Introduction to SQL	9
1.1 What is SQL?	9
1.2 Why Learn SQL?	9
1.3 Databases and Database Types	9
1.4 SQL Command Categories	10
1.5 Core Components: Tables, Keys, Constraints	10
1.6 SQL Coding Style	10
1.7 Environment Setup (PostgreSQL)	11
Module 2: Querying Data (SELECT)	12
2.1 SELECT and FROM	12
Module 3: Data Definition Language (DDL)	16
3.1 CREATE	16
3.2 ALTER	17
3.3 DROP vs. TRUNCATE vs. DELETE	18
Module 4: Data Manipulation Language (DML)	19
4.1 INSERT	19
4.2 UPDATE	19
4.3 DELETE	20
Module 5: Filtering Data	21
5.1 Comparison Operators and NULL	21

5.2 Logical Operators and Precedence	21
5.3 BETWEEN	21
5.4 IN	22
5.5 LIKE, ILIKE, and Regular Expressions	22
Module 6: Combining Data	24
6.1 Basic Joins	24
INNER JOIN	24
LEFT JOIN (LEFT OUTER JOIN)	24
RIGHT JOIN and FULL JOIN	24
CROSS JOIN	25
6.2 Advanced (Anti) Joins	25
6.3 Choosing the Right Join	26
6.4 Joining Multiple Tables	27
6.5 Set Operators	27
Module 7: Row-Level Functions	29
7.1 String Functions	29
7.2 Number Functions	29
7.3 Date & Time Functions	30
7.4 NULL-Handling Functions	30
7.5 CASE — Conditional Logic in SQL	30
Module 8: Aggregation and Analytical (Window) Functions	32
8.1 Aggregate Functions	32
Grouped aggregates	32
8.2 Window Function Basics	33
8.3 Window Aggregate Functions	33
8.4 Window Ranking Functions	34
8.5 Window Value Functions	35
Module 9: Advanced SQL Techniques	37
9.1 Subqueries	37
9.2 Common Table Expressions (CTEs)	38
Recursive CTEs	38
9.3 Views	39
9.4 CTAS and Temporary Tables	39
9.5 Stored Procedures	40
9.6 Triggers	41

Module 10: Performance Optimization	42
10.1 Indexes	42
10.2 Partitions	42
10.3 Reading Query Plans: EXPLAIN and EXPLAIN ANALYZE	43
10.4 Identifying and Fixing Slow Queries	43
10.5 Database-Specific Optimizations	44
 Module 11: AI & SQL	 45
11.1 Generating SQL Queries with AI	45
11.2 AI-Assisted Query Optimization	45
11.3 Automating SQL Code Reviews	46
11.4 Integrating SQL with AI/ML Pipelines	46
11.5 Practical Use Cases	46
 Module 12: SQL Projects	 47
Project 1: Data Warehousing (Data-Engineering Focus)	47
1. Design the warehouse schema	47
2. Build the ELT pipeline (SQL)	48
3. Optimize for analytical workloads	48
Project 2: Exploratory Data Analysis (Analytics/Data-Science Focus)	49
1. Data cleaning and preprocessing	49
2. Statistical analysis: distributions and correlations	49
3. Cohort analysis and time-series trends	50
Project 3: Advanced Analytics	51
A/B testing analysis	51
Churn prediction (feature table for SQL + ML)	51
Real-time analytics with streaming data	51
 Next Steps by Role	 53
Additional Resources	54
Documentation	54
Practice Platforms	54
Further Reading	54

Course Overview

This coursebook takes you from your very first `SELECT` statement to writing production-grade, performance-tuned SQL used by professional data engineers, analysts, and data scientists. It is organised into three progressive levels — Basics, Intermediate, and Advanced — plus two specialised tracks (one weighted toward Data Engineering, one toward Analytics and Data Science) and a closing module on AI-assisted SQL and hands-on projects.

Every module pairs a theory section (what the concept is and why it matters) with a practical section (how it is used, written against a single running e-commerce dataset). Rather than jumping between disconnected toy examples, you will use the same seven tables throughout the whole book, so that by the final chapter you are comfortable navigating a realistic schema the way you would in a real job.

How to Use This Book

- Read the theory first, then type out — do not just read — every practical example in a real PostgreSQL instance.
- Each module ends with Practice Tasks. Attempt them before checking the discussion in the following module.
- Modules 8 and 9 are written with Data Engineers primarily in mind; Modules 7 and 12 (EDA) lean toward Analysts and Data Scientists — but every reader should work through all twelve modules, since the split simply reflects which skills you will use daily.
- SQL keywords appear in UPPERCASE by convention in this book; PostgreSQL itself is case-insensitive to keywords.

Course Map

Level	Modules	Focus	Primary Audience
Basics	1 – 4	Foundations: <code>SELECT</code> , <code>DDL</code> , <code>DML</code>	Everyone
Intermediate	5 – 6	Filtering & combining data	Everyone
Data Engineering	7 – 8	Row-level & window/aggregate functions	Data Engineers
Analytics/DS	7 – 8 (applied)	Insight extraction, statistics via SQL	Analysts, Scientists
Advanced	9 – 10	Subqueries, CTEs, views, performance	All roles
Modern SQL	11	AI & SQL	All roles
Capstone	12	Three end-to-end projects	All roles

The Case Study: NorthStar Commerce

NorthStar Commerce is a mid-sized online retailer selling electronics, home goods, and apparel. Every practical example, exercise, and project in this book queries the same seven-table schema below. Set it up once in a local PostgreSQL database (see Module 1) and you will reuse it for the rest of the course.

Entity-Relationship Summary

The schema models a typical operational e-commerce store: customers browse a catalog organised into categories, place orders composed of line items, pay for those orders, and leave reviews on products they have purchased.

categories

Column	Type	Notes
category_id	INT, PK	Surrogate key
category_name	VARCHAR(100)	e.g. 'Electronics', 'Apparel'
parent_category_id	INT, FK → categories	NULL for top-level categories

products

Column	Type	Notes
product_id	INT, PK	Surrogate key
product_name	VARCHAR(200)	
category_id	INT, FK → categories	
price	NUMERIC(10,2)	Current selling price
cost	NUMERIC(10,2)	Unit cost to NorthStar
stock_quantity	INT	Units on hand
created_at	DATE	Date added to catalog

customers

Column	Type	Notes
customer_id	INT, PK	Surrogate key
first_name	VARCHAR(50)	
last_name	VARCHAR(50)	
email	VARCHAR(150)	Unique
country	VARCHAR(50)	
signup_date	DATE	

orders

Column	Type	Notes
order_id	INT, PK	Surrogate key
customer_id	INT, FK → customers	
order_date	TIMESTAMP	
status	VARCHAR(20)	'placed','shipped','delivered','cancelled'
shipping_country	VARCHAR(50)	

order_items

Column	Type	Notes
order_item_id	INT, PK	Surrogate key
order_id	INT, FK → orders	
product_id	INT, FK → products	
quantity	INT	
unit_price	NUMERIC(10,2)	Price at time of sale

payments

Column	Type	Notes
payment_id	INT, PK	Surrogate key
order_id	INT, FK → orders	
payment_date	TIMESTAMP	
amount	NUMERIC(10,2)	
payment_method	VARCHAR(20)	'card','paypal','bank_transfer'

reviews

Column	Type	Notes
review_id	INT, PK	Surrogate key
product_id	INT, FK → products	
customer_id	INT, FK → customers	
rating	INT	1-5
review_date	DATE	

Why this schema?

It is small enough to hold in your head but rich enough to demonstrate every concept in this book:

one-to-many relationships (customers → orders), **many-to-many relationships** (customers ↔ products via orders), **self-referencing hierarchy** (categories → parent_category_id), and **transactional facts suited to aggregation and window functions** (order_items, payments).

Module 1: Introduction to SQL

1.1 What is SQL?

SQL (Structured Query Language) is the standard declarative language for defining, manipulating, and querying data held in a relational database.

'Declarative' means you describe what result you want, not the step-by-step procedure for getting it — the database engine's query planner decides how to execute your request efficiently.

SQL vs. NoSQL: **relational (SQL) databases** store data in structured tables with fixed schemas and enforce relationships through keys, making them ideal when data integrity and complex joins matter — such as NorthStar Commerce's orders and payments. **NoSQL databases (document, key-value, wide-column, graph stores)** trade some of that structure for flexible schemas and horizontal scalability, which suits high-volume, loosely-structured or rapidly evolving data such as clickstream logs or product catalogs with wildly varying attributes.

1.2 Why Learn SQL?

- SQL is the **common language of data-driven organisations** — nearly every analytics tool, BI dashboard, and ML feature pipeline ultimately reads from or writes to SQL databases.
- It **underpins the daily work of Data Engineers** (building ETL/ELT pipelines and warehouses), **Data Analysts** (answering business questions), and **Data Scientists** (assembling training datasets, feature engineering).
- In modern data stacks, **SQL sits at the centre of ETL** (Extract-Transform-Load, transform before loading), **ELT** (Extract-Load-Transform, transform inside the warehouse using SQL), and **Data Warehousing**, where tools like dbt, Airflow, and cloud warehouses (Snowflake, BigQuery, Redshift) are largely orchestrating SQL under the hood.

1.3 Databases and Database Types

A **Relational Database Management System (RDBMS)** organises data into tables of rows and columns linked by keys, and enforces rules (constraints) that keep the data consistent.

Popular RDBMSs include PostgreSQL, MySQL, SQL Server, and cloud-native analytical warehouses such as BigQuery and Snowflake — the last two relax some traditional RDBMS constraints in favour of massive parallel scan performance.

OLTP (Online Transaction Processing) systems — like the live NorthStar Commerce checkout database — are optimised for many small, fast reads and writes (placing an order, updating stock).

OLAP (Online Analytical Processing) systems — like a NorthStar data warehouse — are optimised for large, complex read queries across millions of historical rows (quarterly revenue by category). Data engineers frequently build pipelines that move data from OLTP systems into OLAP systems.

1.4 SQL Command Categories

Category	Stands For	Purpose	Example Commands
DDL	Data Definition Language	Define/alter structure	CREATE, ALTER, DROP
DML	Data Manipulation Language	Change data	INSERT, UPDATE, DELETE
DQL	Data Query Language	Read data	SELECT
DCL	Data Control Language	Manage permissions	GRANT, REVOKE
TCL	Transaction Control Language	Manage transactions	COMMIT, ROLLBACK, SAVEPOINT

1.5 Core Components: Tables, Keys, Constraints

A **table** stores records (rows) of a single entity type, each described by a fixed set of columns (fields).

A **Primary Key (PK)** uniquely identifies each row in a table — NorthStar's `customer_id`, for instance.

A **Foreign Key (FK)** is a column in one table that references the primary key of another, enforcing referential integrity — `order_items.product_id` must exist in `products.product_id`.

An **index** is a separate structure that speeds up lookups on a column, at the cost of extra storage and slower writes (covered fully in Module 10).

1.6 SQL Coding Style

Readable SQL is maintainable SQL. This book follows conventions typical of professional data teams:

- snake_case for table and column names (`order_items`, not `OrderItems`) — the dominant convention in PostgreSQL and most modern warehouses.
- SQL keywords in UPPERCASE (`SELECT`, `WHERE`, `JOIN`) to visually separate syntax from identifiers.
- One clause per line for anything beyond a trivial query, with `JOIN` conditions and `WHERE` predicates indented.
- Meaningful table aliases (`o` for `orders`, `oi` for `order_items`) rather than single unrelated letters.

```
-- Readable style
SELECT
  c.customer_id,
  c.first_name,
```

```

    o.order_id,
    o.order_date
FROM customers AS c
JOIN orders AS o
    ON o.customer_id = c.customer_id
WHERE o.status = 'delivered'
ORDER BY o.order_date DESC;

```

1.7 Environment Setup (PostgreSQL)

You can install PostgreSQL locally, run it in Docker, or use a free-tier cloud instance. The Docker route is the fastest way to get a clean, disposable environment for working through this book.

Practical: Spin up PostgreSQL with Docker

```

docker run --name northstar-pg \
  -e POSTGRES_PASSWORD=learnsql \
  -e POSTGRES_DB=northstar \
  -p 5432:5432 -d postgres:16

# Connect with the psql CLI
psql -h localhost -U postgres -d northstar

```

Alternatively, install pgAdmin (a graphical client) to browse tables and run queries visually, or use the DBeaver universal client which works with PostgreSQL, MySQL, and SQL Server alike.

Practical: Your first query

```

SELECT NOW();           -- current timestamp
SELECT version();       -- confirm PostgreSQL version

```

Practice Tasks

- 1) **Install PostgreSQL** (locally or via Docker) and connect with psql or pgAdmin.
- 2) In your own words, explain to a colleague the difference between **OLTP** and **OLAP** using NorthStar Commerce as the example.
- 3) List which of NorthStar's seven tables would need foreign keys, and to which parent tables they would point.

Module 2: Querying Data (SELECT)

The **SELECT** statement is the workhorse of SQL — it is how you retrieve and shape data from one or more tables.

This module builds the SELECT statement clause by clause, using NorthStar Commerce data throughout.

2.1 SELECT and FROM

SELECT lists the columns you want returned; **FROM** names the source table.

*Selecting only the columns you need (rather than **SELECT ***) is a best practice in production code: it is more efficient, more readable, and won't silently break when the table gains new columns.*

```
-- All columns (fine for exploration, avoid in production code)
SELECT * FROM products;

-- Specific columns
SELECT product_name, price, stock_quantity
FROM products;
```

Table aliases and self-joins

An **alias** gives a table (or column) a short temporary name for the duration of a query, using **AS** (often omitted).

Aliases are essential once a query involves more than one table, and required when a table is joined to itself — for example, matching each category to its parent category.

```
-- Self-join: list each category with its parent category name
SELECT
    child.category_name AS category,
    parent.category_name AS parent_category
FROM categories AS child
LEFT JOIN categories AS parent
    ON child.parent_category_id = parent.category_id;
```

2.2 WHERE — Filtering Rows

WHERE removes rows before any grouping or selection happens, based on a boolean condition.

*Multiple conditions combine with **AND**, **OR**, and **NOT**; use parentheses to make precedence explicit.*

```
-- Orders placed in 2026 that are NOT cancelled
SELECT order_id, customer_id, order_date, status
```

2.3 ORDER BY – Sorting Results

ORDER BY sorts the final result set; ASC (default) sorts low-to-high, DESC sorts high-to-low. You can sort by multiple columns – later columns break ties in earlier ones.

```
--Highest value customers first;ties broken by most recent signup
SELECT customer_id, first_name, last_name, signup_date
FROM customers
ORDER BY country ASC, signup_date DESC
```

2.4 GROUP BY and Aggregation

GROUP BY collapses multiple rows sharing the same value(s) into one row per group, typically paired with aggregate functions like COUNT, SUM, or AVG.

This is different from ORDER BY, which only reorders rows without combining them.

```
-- Number of orders and total revenue per country
SELECT
    shipping_country,
    COUNT(*)          AS order_count,
    SUM(o.status = 'delivered') -- illustrative only, see Module 7 for CASE
FROM orders AS o
GROUP BY shipping_country;
```

Note:

The illustrative **SUM(o.status = 'delivered')** line above is a preview – proper conditional aggregation with CASE is covered in Module 7. Aggregate functions themselves are covered fully in Module 8.

2.5 HAVING – Filtering Groups

WHERE filters individual rows before grouping;

HAVING filters entire groups after aggregation. A common mistake is trying to filter on an aggregate inside WHERE – PostgreSQL will reject it because aggregates don't exist yet at that stage of execution.

```
-- Countries with more than 50 orders
SELECT shipping_country, COUNT(*) AS order_count
```

```
FROM orders
GROUP BY shipping_country
HAVING COUNT(*) > 50;
```

2.6 DISTINCT – Removing Duplicates

```
-- Which countries have we ever shipped to?
SELECT DISTINCT shipping_country FROM orders;

-- PostgreSQL-specific: one row per customer, keeping their most recent order
SELECT DISTINCT ON (customer_id) customer_id, order_id, order_date
FROM orders
ORDER BY customer_id, order_date DESC;
```

2.7 LIMIT, OFFSET

LIMIT caps the number of rows returned;

OFFSET skips a number of rows first, enabling pagination.

SQL Server uses *TOP* with different syntax instead of **LIMIT**; PostgreSQL and MySQL both use **LIMIT/OFFSET**.

```
-- Top 5 highest-priced products
SELECT product_name, price
FROM products
ORDER BY price DESC
LIMIT 5;

-- Page 2 of a product listing, 20 per page
SELECT product_name, price
FROM products
ORDER BY product_id
LIMIT 20 OFFSET 20;
```

2.8 Query Order and Execution

SQL is written in one order but the database engine logically processes clauses in another. Understanding this order explains many beginner errors (like referencing a column alias in **WHERE**).

Step	Clause	What Happens
1	FROM / JOIN	Identify and combine source tables
2	WHERE	Filter individual rows
3	GROUP BY	Group remaining rows
4	HAVING	Filter groups
5	SELECT	Compute output columns/expressions
6	ORDER BY	Sort the result
7	LIMIT / OFFSET	Trim the result

Practice Tasks

- 1) Write a query returning the 10 most expensive products in the 'Electronics' category, cheapest first among that top 10.
- 2) Find every country that has placed more than 20 orders.
- 3) Explain why 'SELECT product_name, COUNT(*) AS n FROM products WHERE n > 1' fails, and rewrite it correctly.

Module 3: Data Definition Language (DDL)

DDL statements define and modify the structure of your database — the tables, columns, and constraints that hold your data, as opposed to the data itself.

3.1 CREATE

CREATE DATABASE makes a new database;

CREATE SCHEMA groups related tables inside one;

CREATE TABLE defines a new table's columns, data types, and constraints.

Practical: Building the NorthStar Commerce schema (part 1 of 2)

```
CREATE TABLE categories (
    category_id      SERIAL PRIMARY KEY,
    category_name    VARCHAR(100) NOT NULL,
    parent_category_id INT REFERENCES categories(category_id)
);

CREATE TABLE products (
    product_id      SERIAL PRIMARY KEY,
    product_name    VARCHAR(200) NOT NULL,
    category_id     INT REFERENCES categories(category_id),
    price           NUMERIC(10,2) NOT NULL CHECK (price >= 0),
    cost            NUMERIC(10,2) NOT NULL CHECK (cost >= 0),
    stock_quantity  INT NOT NULL DEFAULT 0,
    created_at      DATE NOT NULL DEFAULT CURRENT_DATE
);

CREATE TABLE customers (
    customer_id     SERIAL PRIMARY KEY,
    first_name      VARCHAR(50) NOT NULL,
    last_name       VARCHAR(50) NOT NULL,
    email           VARCHAR(150) UNIQUE NOT NULL,
    country         VARCHAR(50),
    signup_date     DATE NOT NULL DEFAULT CURRENT_DATE
);

CREATE TABLE orders (
    order_id        SERIAL PRIMARY KEY,
    customer_id     INT REFERENCES customers(customer_id),
    order_date      TIMESTAMP NOT NULL DEFAULT NOW(),
    status          VARCHAR(20) NOT NULL DEFAULT 'placed',
    shipping_country VARCHAR(50)
);
```

Practical: Building the NorthStar Commerce schema (part 2 of 2)

```

CREATE TABLE order_items (
    order_item_id SERIAL PRIMARY KEY,
    order_id      INT REFERENCES orders(order_id),
    product_id    INT REFERENCES products(product_id),
    quantity      INT NOT NULL CHECK (quantity > 0),
    unit_price    NUMERIC(10,2) NOT NULL
);

CREATE TABLE payments (
    payment_id     SERIAL PRIMARY KEY,
    order_id       INT REFERENCES orders(order_id),
    payment_date   TIMESTAMP NOT NULL DEFAULT NOW(),
    amount         NUMERIC(10,2) NOT NULL,
    payment_method VARCHAR(20)
);

CREATE TABLE reviews (
    review_id      SERIAL PRIMARY KEY,
    product_id     INT REFERENCES products(product_id),
    customer_id    INT REFERENCES customers(customer_id),
    rating         INT CHECK (rating BETWEEN 1 AND 5),
    review_date    DATE NOT NULL DEFAULT CURRENT_DATE
);

```

CREATE TABLE AS (CTAS) creates a new table populated with the result of a query — useful for snapshots or derived tables (covered in depth in Module 9).

```

-- Snapshot of currently low-stock products
CREATE TABLE low_stock_snapshot AS
SELECT product_id, product_name, stock_quantity
FROM products
WHERE stock_quantity < 10;

```

3.2 ALTER

ALTER TABLE modifies an existing table's structure without dropping and recreating it — adding a column, changing a data type, adding or dropping a constraint.

```
-- Add a loyalty tier to customers
ALTER TABLE customers ADD COLUMN loyalty_tier VARCHAR(20) DEFAULT 'standard';

-- Rename a column
ALTER TABLE products RENAME COLUMN stock_quantity TO units_in_stock;

-- Widen a column's data type
ALTER TABLE products ALTER COLUMN product_name TYPE VARCHAR(300);

-- Drop a column
ALTER TABLE customers DROP COLUMN loyalty_tier;
```

3.3 DROP vs. TRUNCATE vs. DELETE

These three commands all remove data or objects but behave very differently, a distinction that trips up many beginners and matters a great deal in production.

Command	Removes	Structure Kept?	Rollback?	Speed
DROP TABLE	Table + all data + structure	No — table is gone	Within a transaction, yes	Instant
TRUNCATE TABLE	All rows	Yes — empty table remains	Within a transaction, yes	Very fast (no row-by-row logging)
DELETE FROM ... [WHERE]	Matching rows (or all, if no WHERE)	Yes	Yes	Slower — row-by-row, can use WHERE

```
DELETE FROM order_items WHERE order_id = 10231; -- selective, logged row by row
TRUNCATE TABLE low_stock_snapshot;             -- wipe a staging table fast
DROP TABLE low_stock_snapshot;                 -- remove the table entirely
```

Practice Tasks

- 1) Write the **CREATE TABLE** statement for a new 'shipments' table tracking order_id, carrier, tracking_number, and shipped_date.
- 2) Add a 'discontinued' BOOLEAN column to products with a default of FALSE.
- 3) Explain, in a sentence each, when you would choose DELETE over TRUNCATE.

Module 4: Data Manipulation Language (DML)

DML statements change the data inside tables: **INSERT** adds rows, **UPDATE** modifies existing rows, and **DELETE** removes rows.

4.1 INSERT

```
-- Insert a single row, naming columns explicitly (recommended)
INSERT INTO categories (category_name, parent_category_id)
VALUES ('Electronics', NULL);

-- Insert multiple rows in one statement
INSERT INTO categories (category_name, parent_category_id)
VALUES
    ('Laptops', 1),
    ('Headphones', 1),
    ('Apparel', NULL);

-- Insert rows selected from another table (bulk load pattern)
INSERT INTO low_stock_snapshot (product_id, product_name, stock_quantity)
SELECT product_id, product_name, stock_quantity
FROM products
WHERE stock_quantity < 5;
```

For very large bulk loads, PostgreSQL's **COPY** command (loading directly from a CSV/file) is dramatically faster than many individual **INSERT** statements — a detail data engineers rely on heavily when populating a warehouse.

4.2 UPDATE

UPDATE modifies existing rows and should almost always be paired with a **WHERE** clause — an **UPDATE** without **WHERE** changes every row in the table.

```
-- Apply a 10% price increase to one product
UPDATE products
SET price = price * 1.10 WHERE product_id = 205;

-- Update multiple columns at once
UPDATE orders
SET status = 'shipped',
    shipping_country = 'KE'
WHERE order_id = 10231;
```

```
-- Conditional update across many rows
UPDATE products
SET stock_quantity = stock_quantity - 1
WHERE product_id IN (
    SELECT product_id FROM order_items WHERE order_id = 10231
);
```

4.3 DELETE

```
-- Remove a single cancelled order's line items first (FK constraint), then the order
DELETE FROM order_items WHERE order_id = 10450;
DELETE FROM orders WHERE order_id = 10450;
```

DELETE vs. TRUNCATE was compared in Module 3. Cascading deletes let a database automatically remove dependent rows when a parent row is deleted, using **ON DELETE CASCADE** on the foreign key definition — convenient, but dangerous if applied carelessly, since it can silently remove far more data than intended.

```
-- Re-define order_items so deleting an order cleans up its line items
automatically
ALTER TABLE order_items
    DROP CONSTRAINT order_items_order_id_fkey,
    ADD CONSTRAINT order_items_order_id_fkey
        FOREIGN KEY (order_id) REFERENCES orders(order_id) ON DELETE CASCADE;
```

Safety habit

Before running an UPDATE or DELETE in production, run the equivalent SELECT with the same WHERE clause first, to confirm exactly which rows will be affected. Many teams also require these statements to run inside an **explicit transaction (BEGIN ... COMMIT/ROLLBACK)** so a mistake can be undone before it is committed.

Practice Prompts

- 1) Insert three new customers into NorthStar Commerce using a single INSERT statement.
- 2) Write an UPDATE that marks every order older than 30 days and still 'placed' as 'delivered'.
- 3) Explain what would happen if you ran DELETE FROM orders; with no WHERE clause, and how ON DELETE CASCADE would change that.

Module 5: Filtering Data

This module goes deeper into WHERE-clause filtering: comparison and logical operators, ranges, list membership, and pattern matching.

5.1 Comparison Operators and NULL

Standard comparison operators are =, !=/<>, <, >, <=, >=. NULL represents 'unknown' rather than a value, so it can never be compared with = or != — you must use IS NULL or IS NOT NULL.

```
-- Orders with no shipping country recorded yet
SELECT order_id, customer_id
FROM orders
WHERE shipping_country IS NULL;

-- Products that DO have a recorded cost
SELECT product_name FROM products WHERE cost IS NOT NULL;
```

5.2 Logical Operators and Precedence

AND binds tighter than **OR**, so mixing them without parentheses can silently change your query's meaning. Always use parentheses to make intent explicit.

```
-- Ambiguous without parentheses: does NOT mean what you might expect
SELECT * FROM orders
WHERE status = 'placed' OR status = 'shipped' AND shipping_country = 'KE';

-- Explicit and correct: placed anywhere, OR shipped specifically to Kenya
SELECT * FROM orders
WHERE status = 'placed' OR (status = 'shipped' AND shipping_country = 'KE');
```

5.3 BETWEEN

BETWEEN filters an inclusive range and is more readable than chaining >= and <= manually, though the two are functionally equivalent.

```
-- Mid-priced products
SELECT product_name, price
FROM products
WHERE price BETWEEN 20 AND 100; -- inclusive of 20 and 100

-- Orders placed in Q1 2026
SELECT order_id, order_date
FROM orders
WHERE order_date BETWEEN '2026-01-01' AND '2026-03-31';
```

5.4 IN

IN checks membership against a fixed list of values, and is cleaner than a long chain of OR conditions. It also accepts a subquery, letting you filter against a dynamically computed set of values.

```
-- IN vs. chained OR (equivalent results)
SELECT * FROM orders WHERE status IN ('placed', 'shipped');
SELECT * FROM orders WHERE status = 'placed' OR status = 'shipped';

-- Subquery inside IN: customers who have left a 5-star review
SELECT first_name, last_name
FROM customers
WHERE customer_id IN (
    SELECT customer_id FROM reviews WHERE rating = 5
);
```

5.5 LIKE, ILIKE, and Regular Expressions

LIKE performs case-sensitive pattern matching using % (any sequence of characters) and _ (any single character).

ILIKE is PostgreSQL's case-insensitive variant. PostgreSQL also supports POSIX regular expressions via ~ (matches, case-sensitive) and ~* (matches, case-insensitive).

```
-- Products with 'wireless' anywhere in the name, case-insensitive
SELECT product_name FROM products WHERE product_name ILIKE '%wireless%';

-- Emails ending in a specific domain
SELECT email FROM customers WHERE email LIKE '%@gmail.com';

-- Regex: product names starting with a digit (e.g. '4K Monitor')
SELECT product_name FROM products WHERE product_name ~ '^[0-9]';
```

Practice Tasks

- 1) Find all customers who signed up between two dates of your choosing using **BETWEEN**.
- 2) Find products priced under 20 **OR** over 500, but only within the 'Electronics' category — write it with correctly-placed parentheses.
- 3) Find every product whose name contains 'pro' regardless of case, using **ILIKE**.

Module 6: Combining Data

Real questions rarely live inside a single table. This module covers joins — the primary tool for combining rows across related tables — plus set operators for combining entire result sets.

6.1 Basic Joins

A **join** combines rows from two (or more) tables based on a related column, typically a foreign key matching a primary key.

INNER JOIN

Returns only rows that have a match in both tables. This is the default and most common join.

```
-- Every order line together with the product name and category
SELECT
    o.order_id,
    p.product_name,
    oi.quantity,
    oi.unit_price
FROM orders AS o
INNER JOIN order_items AS oi ON oi.order_id = o.order_id
INNER JOIN products AS p ON p.product_id = oi.product_id;
```

LEFT JOIN (LEFT OUTER JOIN)

Returns every row from the left table, and matching rows from the right table where they exist — unmatched right-side columns come back as NULL.

Ideal for 'find everything, plus what relates to it if anything.'

```
-- Every customer, and their orders if they have any (including zero-order
customers)
SELECT
    c.customer_id,
    c.first_name,
    o.order_id
FROM customers AS c
LEFT JOIN orders AS o ON o.customer_id = c.customer_id;
```

RIGHT JOIN and FULL JOIN

RIGHT JOIN mirrors LEFT JOIN, keeping all rows from the right table instead — it is rarely used in practice because you can always rewrite it as a LEFT JOIN by swapping table order. FULL JOIN (FULL OUTER JOIN) keeps all rows from both sides, matching where possible and filling NULLs elsewhere.

```
-- Every product and every review, matched where they relate, gaps kept on both
sides
SELECT
    p.product_name,
    r.rating
FROM products AS p
FULL JOIN reviews AS r ON r.product_id = p.product_id;
```

CROSS JOIN

Produces the Cartesian product — every row of one table paired with every row of the other. Rarely used on real business data, but useful for generating combinations, such as every product against every possible discount tier for a promotion simulation.

```
SELECT p.product_name, d.discount_pct
FROM products AS p
CROSS JOIN (VALUES (0.05), (0.10), (0.20)) AS d(discount_pct);
```

6.2 Advanced (Anti) Joins

An anti-join finds rows in one table that have no match in another — a very common analytical question ('customers who have never ordered').

```
-- Anti-join via LEFT JOIN + WHERE
SELECT c.customer_id, c.first_name
FROM customers AS c
LEFT JOIN orders AS o ON o.customer_id = c.customer_id
WHERE o.order_id IS NULL;

-- Equivalent using NOT EXISTS (often the better-performing choice)
SELECT c.customer_id, c.first_name
FROM customers AS c
WHERE NOT EXISTS (
    SELECT 1 FROM orders AS o WHERE o.customer_id = c.customer_id
);

-- Equivalent using NOT IN -- CAUTION: breaks silently if the subquery can return
NULL
SELECT customer_id, first_name
FROM customers
WHERE customer_id NOT IN (
    SELECT customer_id FROM orders WHERE customer_id IS NOT NULL
);
```

A semi-join answers 'does a match exist?' without pulling any columns from the other table — expressed with EXISTS or IN.

```
-- Semi-join: customers who have placed at least one order
SELECT c.customer_id, c.first_name
FROM customers AS c
WHERE EXISTS (
    SELECT 1 FROM orders AS o WHERE o.customer_id = c.customer_id
);
```

6.3 Choosing the Right Join

Ask two questions:

(1) do I need rows even when there's no match on the other side? and (2) do I need columns from the other table, or just to know whether a match exists?

INNER JOIN when you only want matched pairs;

LEFT JOIN when the left side must be fully preserved;

EXISTS/NOT EXISTS when you only care about match existence, not the matched columns — these often perform better than an equivalent JOIN with DISTINCT.

Question	Use
Only rows present in both tables	INNER JOIN
All of table A, matched data from B where available	LEFT JOIN
Rows in A with NO match in B	LEFT JOIN ... WHERE B IS NULL, or NOT EXISTS
Rows in A that DO have a match in B (columns not needed)	EXISTS / IN
Every combination of A and B	CROSS JOIN
Everything from both sides, matched or not	FULL JOIN

6.4 Joining Multiple Tables

```
-- Chain three joins: order -> line item -> product -> category
SELECT
    o.order_id,
    cat.category_name,
    p.product_name,
    oi.quantity
FROM orders AS o
JOIN order_items AS oi ON oi.order_id = o.order_id
JOIN products AS p ON p.product_id = oi.product_id
JOIN categories AS cat ON cat.category_id = p.category_id;
```

Self-joins (introduced in Module 2 for categories) and non-equijoins — joins using an operator other than =, such as matching orders to a promotional period by date range — both remain valid ON conditions.

```
-- Non-equijoin: match each order to the promo campaign active on its order_date
SELECT o.order_id, promo.campaign_name
FROM orders AS o
JOIN promotions AS promo
    ON o.order_date BETWEEN promo.start_date AND promo.end_date;
```

6.5 Set Operators

Set operators combine the results of two SELECT statements with the same number and type of columns, rather than combining columns side-by-side like a join.

Operator	Behaviour
UNION	Combines rows from both queries, removing duplicates
UNION ALL	Combines rows from both queries, keeping duplicates (faster — no dedup step)
INTERSECT	Only rows present in BOTH result sets
EXCEPT (MINUS in some DBs)	Rows in the first result set that are NOT in the second

```
-- Everyone who is either a customer OR has left a review, deduplicated
SELECT customer_id FROM customers
UNION
SELECT customer_id FROM reviews;
```

```
-- Customers who bought something in BOTH January and February 2026
SELECT DISTINCT o.customer_id FROM orders o
WHERE o.order_date BETWEEN '2026-01-01' AND '2026-01-31'
INTERSECT
SELECT DISTINCT o.customer_id FROM orders o
WHERE o.order_date BETWEEN '2026-02-01' AND '2026-02-28';

-- Customers who ordered in January but did NOT order in February
SELECT DISTINCT o.customer_id FROM orders o
WHERE o.order_date BETWEEN '2026-01-01' AND '2026-01-31'
EXCEPT
SELECT DISTINCT o.customer_id FROM orders o
WHERE o.order_date BETWEEN '2026-02-01' AND '2026-02-28';
```

Practice Tasks

- 1) Write a query listing every product that has never been ordered (an anti-join).
- 2) Rewrite that query three ways: `LEFT JOIN + WHERE NULL`, `NOT EXISTS`, and `NOT IN` — and explain the NULL trap with `NOT IN`.
- 3) Using `UNION ALL` vs `UNION`, explain in your own words why `UNION ALL` is typically faster.

Module 7: Row-Level Functions

Row-level (scalar) functions transform a single value into another single value, one row at a time — as opposed to aggregate functions, which collapse many rows into one (Module 8).

7.1 String Functions

```
SELECT
    UPPER(first_name)                AS shout_name,
    LOWER(email)                    AS clean_email,
    INITCAP(product_name)           AS title_case_name,
    LENGTH(product_name)            AS name_length,
    SUBSTRING(email FROM 1 FOR POSITION('a' IN email) - 1) AS username,
    TRIM(product_name)              AS trimmed_name,
    first_name || ' ' || last_name   AS full_name,           -- concatenation
    CONCAT(first_name, ' ', last_name) AS full_name_alt,
    REPLACE(product_name, 'Wireless', 'Wired') AS renamed,
    SPLIT_PART(email, 'a', 2)        AS email_domain,
    REGEXP_REPLACE(product_name, '[0-9]+', '', 'g') AS name_no_digits
FROM customers, products
LIMIT 5;
```

7.2 Number Functions

```
SELECT
    price,
    ABS(price - cost)                AS price_cost_gap,
    ROUND(price * 1.16, 2)          AS price_with_16pct_vat,
    CEIL(price)                     AS price_rounded_up,
    FLOOR(price)                    AS price_rounded_down,
    POWER(quantity, 2)              AS quantity_squared,
    SQRT(price)                     AS price_sqrt,
    price % 10                      AS mod_by_ten,
    price::INT                      AS price_as_int,           -- PostgreSQL cast
    shorthand
    CAST(price AS INT)              AS price_as_int_ansi      -- ANSI-standard
    equivalent
FROM order_items
LIMIT 5;
```

7.3 Date & Time Functions

```
SELECT
    order_date,
    EXTRACT(YEAR FROM order_date)           AS order_year,
    DATE_PART('month', order_date)          AS order_month,
    DATE_TRUNC('month', order_date)         AS order_month_start,
    AGE(NOW(), order_date)                  AS time_since_order,
    order_date + INTERVAL '7 days'          AS follow_up_date,
    TO_CHAR(order_date, 'YYYY-MM-DD')       AS order_date_text,
    TO_DATE('2026-06-01', 'YYYY-MM-DD')    AS parsed_date,
    CURRENT_DATE                            AS today,
    order_date AT TIME ZONE 'UTC' AT TIME ZONE 'Africa/Nairobi' AS order_time_nairobi
FROM orders
LIMIT 5;
```

7.4 NULL-Handling Functions

COALESCE returns the first non-NULL argument from a list — the most common way to supply defaults.

NULLIF returns NULL if two expressions are equal (handy for avoiding division-by-zero).

```
-- Default missing shipping country to 'Unknown'
SELECT order_id, COALESCE(shipping_country, 'Unknown') AS shipping_country
FROM orders;

-- Avoid a divide-by-zero when computing an average order value per item count
SELECT
    order_id,
    total_amount / NULLIF(item_count, 0) AS avg_item_value
FROM order_summary; -- a derived table, see Module 9's CTAS/CTE examples
```

7.5 CASE — Conditional Logic in SQL

CASE evaluates conditions and returns different values accordingly.

'Simple CASE' compares one expression against several values; 'searched CASE' evaluates independent boolean conditions. CASE can be used inside SELECT, WHERE, ORDER BY, and even inside aggregate functions for conditional aggregation.

```

-- Simple CASE: translate payment_method codes into readable labels
SELECT
    payment_method,
    CASE payment_method
        WHEN 'card' THEN 'Credit/Debit Card'
        WHEN 'paypal' THEN 'PayPal'
        WHEN 'bank_transfer' THEN 'Bank Transfer'
        ELSE 'Other'
    END AS payment_method_label
FROM payments;

-- Searched CASE: bucket products into price tiers
SELECT
    product_name,
    price,
    CASE
        WHEN price < 20 THEN 'Budget'
        WHEN price BETWEEN 20 AND 100 THEN 'Mid-range'
        ELSE 'Premium'
    END AS price_tier
FROM products
ORDER BY
    CASE
        WHEN price < 20 THEN 1
        WHEN price BETWEEN 20 AND 100 THEN 2
        ELSE 3
    END;

-- CASE inside an aggregate: conditional counting (revisits Module 2's preview)
SELECT
    shipping_country,
    COUNT(*) AS total_orders,
    SUM(CASE WHEN status = 'delivered' THEN 1 ELSE 0 END) AS delivered_orders,
    SUM(CASE WHEN status = 'cancelled' THEN 1 ELSE 0 END) AS cancelled_orders
FROM orders
GROUP BY shipping_country;

```

Practice Tasks

- 1) Write a query that extracts each customer's email username and domain into two separate columns.
- 2) Bucket every order into 'Same week', 'Same month', or 'Older' relative to today using CASE and date arithmetic.
- 3) Use COALESCE to report 0 instead of NULL for products that have never been reviewed (hint: you will need a LEFT JOIN and aggregation from Module 8).

Module 8: Aggregation and Analytical (Window) Functions

This module is central to Data Engineering and data-pipeline work: computing summary metrics (aggregate functions) and row-by-row analytics that still see the surrounding rows (window functions) — the backbone of reporting tables, dashboards, and feature engineering.

8.1 Aggregate Functions

Aggregate functions collapse a set of rows into a single summary value: `COUNT`, `SUM`, `AVG`, `MIN`, `MAX` are the five you will use constantly.

```
SELECT
  COUNT(*)           AS total_order_lines,
  COUNT(DISTINCT order_id) AS distinct_orders,
  SUM(quantity * unit_price) AS total_revenue,
  AVG(unit_price)      AS avg_unit_price,
  MIN(unit_price)      AS cheapest_line,
  MAX(unit_price)      AS priciest_line
FROM order_items;
```

`COUNT(*)` counts every row, including ones with NULLs anywhere; `COUNT(column)` counts only rows where that specific column is non-NULL — a subtle but important difference.

```
-- Every order (COUNT(*)) vs only orders where shipping_country is known
SELECT
  COUNT(*)           AS all_orders,
  COUNT(shipping_country) AS orders_with_known_country
FROM orders;
```

Grouped aggregates

```
-- Revenue and order count per product category
SELECT
  cat.category_name,
  COUNT(DISTINCT o.order_id) AS orders,
  SUM(oi.quantity * oi.unit_price) AS revenue
FROM orders AS o
JOIN order_items AS oi ON oi.order_id = o.order_id
JOIN products AS p ON p.product_id = oi.product_id
JOIN categories AS cat ON cat.category_id = p.category_id
GROUP BY cat.category_name
ORDER BY revenue DESC;
```

8.2 Window Function Basics

A **window function** performs a calculation across a set of rows related to the current row — the 'window' — WITHOUT collapsing the result into one row per group, unlike GROUP BY.

This makes it possible to show a running total, rank, or group average right alongside the original row-level detail.

```
SELECT
  order_id,
  customer_id,
  order_date,
  total_amount,
  AVG(total_amount) OVER ()           AS overall_avg_order,
  AVG(total_amount) OVER (PARTITION BY customer_id) AS customer_avg_order
FROM order_summary;  -- one row per order, with total_amount pre-computed
```

PARTITION BY divides rows into groups the window function is calculated within (similar in spirit to GROUP BY, but without collapsing rows); an ORDER BY inside OVER() controls the row sequence used for running/ranking calculations.

8.3 Window Aggregate Functions

```
-- Running total of revenue per customer, ordered by order date
SELECT
  customer_id,
  order_date,
  total_amount,
  SUM(total_amount) OVER (
    PARTITION BY customer_id
    ORDER BY order_date
    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
  ) AS running_total,
  AVG(total_amount) OVER (
    PARTITION BY customer_id
    ORDER BY order_date
    ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
  ) AS moving_avg_last_3_orders
FROM order_summary
ORDER BY customer_id, order_date;
```

8.4 Window Ranking Functions

Function	Behaviour on Ties
ROW_NUMBER()	Always unique, sequential — ties broken arbitrarily by ORDER BY
RANK()	Ties share a rank; next rank skips (1,1,3,4...)
DENSE_RANK()	Ties share a rank; next rank does NOT skip (1,1,2,3...)
NTILE(n)	Divides rows into n roughly equal buckets
PERCENT_RANK()	Relative rank as a fraction between 0 and 1
CUME_DIST()	Cumulative distribution — fraction of rows with value <= current

```
-- Rank products by revenue within their own category
SELECT
    cat.category_name,
    p.product_name,
    SUM(oi.quantity * oi.unit_price) AS revenue,
    ROW_NUMBER() OVER (PARTITION BY cat.category_name ORDER BY SUM(oi.quantity * oi.unit_price)
DESC) AS row_num,
    RANK() OVER (PARTITION BY cat.category_name ORDER BY SUM(oi.quantity * oi.unit_price)
DESC) AS rnk,
    DENSE_RANK() OVER (PARTITION BY cat.category_name ORDER BY SUM(oi.quantity * oi.unit_price)
DESC) AS dense_rnk,
    NTILE(4) OVER (PARTITION BY cat.category_name ORDER BY SUM(oi.quantity * oi.unit_price)
DESC) AS revenue_quartile
FROM order_items AS oi
JOIN products AS p ON p.product_id = oi.product_id
JOIN categories AS cat ON cat.category_id = p.category_id
GROUP BY cat.category_name, p.product_name;
```

Common use

`SELECT * FROM (... ROW_NUMBER() ... ) AS ranked WHERE row_num <= 3` is the standard pattern for 'top N per group' — e.g. the top 3 best-selling products in each category.

8.5 Window Value Functions

```
-- Compare each order's value to the customer's previous and next order
SELECT
  customer_id,
  order_date,
  total_amount,
  LAG(total_amount) OVER (PARTITION BY customer_id ORDER BY order_date) AS
prev_order_amount,
  LEAD(total_amount) OVER (PARTITION BY customer_id ORDER BY order_date) AS
next_order_amount,
  FIRST_VALUE(total_amount) OVER (
    PARTITION BY customer_id ORDER BY order_date
    ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
  ) AS first_order_amount,
  LAST_VALUE(total_amount) OVER (
    PARTITION BY customer_id ORDER BY order_date
    ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
  ) AS last_order_amount,
  NTH_VALUE(total_amount, 2) OVER (
    PARTITION BY customer_id ORDER BY order_date
    ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
  ) AS second_order_amount
FROM order_summary
ORDER BY customer_id, order_date;
```

LAG/LEAD are the standard way to compute period-over-period change (e.g. month-over-month revenue growth) — a pattern data engineers build into nearly every reporting pipeline.

```
-- Month-over-month revenue growth
WITH monthly_revenue AS (
  SELECT
    DATE_TRUNC('month', o.order_date) AS month,
    SUM(oi.quantity * oi.unit_price) AS revenue
  FROM orders AS o
  JOIN order_items AS oi ON oi.order_id = o.order_id
  GROUP BY 1
)
SELECT
  month,
  revenue,
  LAG(revenue) OVER (ORDER BY month) AS prev_month_revenue,
  ROUND(
    (revenue - LAG(revenue) OVER (ORDER BY month))
    / NULLIF(LAG(revenue) OVER (ORDER BY month), 0) * 100, 1
  ) AS pct_growth
FROM monthly_revenue
ORDER BY month;
```

Practice Tasks

- 1) Find the top 3 highest-spending customers overall using `RANK()`.
- 2) For each product, compute its rank by total units sold within its category using `DENSE_RANK()`.
- 3) Build a query showing each customer's order total alongside the running cumulative total of all their orders to date, ordered by `order_date`.

Module 9: Advanced SQL Techniques

9.1 Subqueries

A **subquery** is a query nested inside another query. It can return a single value (scalar), a single row, a full table of rows/columns, or reference columns from the outer query (correlated).

```
-- Scalar subquery: flag products priced above the overall average
SELECT product_name, price
FROM products
WHERE price > (SELECT AVG(price) FROM products);

-- Row subquery: find the product matching the single most-expensive item's category and price
SELECT product_name
FROM products
WHERE (category_id, price) = (
    SELECT category_id, MAX(price) FROM products WHERE category_id = 3 GROUP BY category_id
);

-- Table subquery: revenue per category, then filter on the derived result
SELECT category_name, revenue
FROM (
    SELECT cat.category_name, SUM(oi.quantity * oi.unit_price) AS revenue
    FROM order_items AS oi
    JOIN products AS p ON p.product_id = oi.product_id
    JOIN categories AS cat ON cat.category_id = p.category_id
    GROUP BY cat.category_name
) AS category_revenue
WHERE revenue > 10000;

-- Correlated subquery: each customer's most recent order date
SELECT
    c.customer_id,
    c.first_name,
    (SELECT MAX(o.order_date) FROM orders AS o WHERE o.customer_id = c.customer_id) AS last_order_date
FROM customers AS c;

-- EXISTS: customers who have reviewed a product they bought (uses correlated subquery)
SELECT c.customer_id, c.first_name
FROM customers AS c
WHERE EXISTS (
    SELECT 1 FROM reviews AS r WHERE r.customer_id = c.customer_id
);
```

Correlated vs. uncorrelated

An **uncorrelated** subquery runs once, independent of the outer query. A **correlated** subquery references a column from the outer query and conceptually re-runs once per outer row — which is powerful, but can be slow on large tables if the optimiser cannot rewrite it into a join internally.

9.2 Common Table Expressions (CTEs)

A **CTE**, defined with **WITH**, names a temporary result set that can be referenced later in the same query — improving readability over deeply nested subqueries, and enabling recursion.

```
-- Single CTE: category revenue, reused for a readable filter
WITH category_revenue AS (
  SELECT cat.category_name, SUM(oi.quantity * oi.unit_price) AS revenue
  FROM order_items AS oi
  JOIN products AS p ON p.product_id = oi.product_id
  JOIN categories AS cat ON cat.category_id = p.category_id
  GROUP BY cat.category_name
)
SELECT * FROM category_revenue WHERE revenue > 10000;

-- Multiple CTEs, chained: monthly revenue, then month-over-month change
WITH monthly_revenue AS (
  SELECT DATE_TRUNC('month', o.order_date) AS month,
         SUM(oi.quantity * oi.unit_price) AS revenue
  FROM orders AS o
  JOIN order_items AS oi ON oi.order_id = o.order_id
  GROUP BY 1
),
revenue_with_change AS (
  SELECT month, revenue,
         revenue - LAG(revenue) OVER (ORDER BY month) AS revenue_change
  FROM monthly_revenue
)
SELECT * FROM revenue_with_change ORDER BY month;
```

Recursive CTEs

A **recursive CTE** repeatedly executes to walk hierarchical or graph-like data — perfect for NorthStar's self-referencing category tree, where a category can have a parent category, which can itself have a parent.

```
-- Walk the full category tree from top-level categories downward
WITH RECURSIVE category_tree AS (
  -- Anchor: top-level categories
  SELECT category_id, category_name, parent_category_id, 1 AS depth
  FROM categories
  WHERE parent_category_id IS NULL

  UNION ALL

  -- Recursive step: find children of the previous level
  SELECT c.category_id, c.category_name, c.parent_category_id, ct.depth + 1
  FROM categories AS c
  JOIN category_tree AS ct ON c.parent_category_id = ct.category_id
)
SELECT * FROM category_tree ORDER BY depth, category_name;
```

9.3 Views

A **view** is a stored, named query that behaves like a virtual table — it always reflects live underlying data. A **materialized view** instead stores the query's result physically on disk and must be refreshed explicitly, trading freshness for query speed.

```
-- Regular view: always up to date, computed on every query
CREATE VIEW order_summary AS
SELECT
    o.order_id,
    o.customer_id,
    o.order_date,
    o.status,
    SUM(oi.quantity * oi.unit_price) AS total_amount,
    COUNT(oi.order_item_id) AS item_count
FROM orders AS o
JOIN order_items AS oi ON oi.order_id = o.order_id
GROUP BY o.order_id, o.customer_id, o.order_date, o.status;

-- Materialized view: snapshot, must be refreshed manually or on a schedule
CREATE MATERIALIZED VIEW monthly_revenue_mv AS
SELECT DATE_TRUNC('month', order_date) AS month, SUM(total_amount) AS revenue
FROM order_summary
GROUP BY 1;

REFRESH MATERIALIZED VIEW monthly_revenue_mv;
```

A view is updatable (you can INSERT/UPDATE through it into the base table) only if it is simple enough — typically a single-table view with no aggregation, DISTINCT, or GROUP BY.

9.4 CTAS and Temporary Tables

CREATE TABLE AS (introduced in Module 3) materialises a query's result as a real, permanent table — useful for one-off snapshots and denormalised reporting tables.

A **temporary table** exists only for the duration of the current session (or transaction) and is automatically dropped afterward, making it ideal for intermediate steps in a complex, multi-stage transformation.

```
-- CTAS: a permanent denormalised reporting table
CREATE TABLE product_performance AS
SELECT
    p.product_id, p.product_name,
    SUM(oi.quantity) AS units_sold,
    SUM(oi.quantity * oi.unit_price) AS revenue,
    AVG(r.rating) AS avg_rating
FROM products AS p
LEFT JOIN order_items AS oi ON oi.product_id = p.product_id
LEFT JOIN reviews AS r ON r.product_id = p.product_id
GROUP BY p.product_id, p.product_name;

-- Temp table: intermediate step in a multi-stage ETL job, gone at session end
CREATE TEMPORARY TABLE tmp_high_value_customers AS
SELECT customer_id, SUM(total_amount) AS lifetime_value
FROM order_summary
GROUP BY customer_id
HAVING SUM(total_amount) > 1000;
```

9.5 Stored Procedures

A **stored procedure** bundles procedural logic (parameters, control flow, possibly multiple statements and transaction control) inside the database itself, callable with CALL.

```
CREATE OR REPLACE PROCEDURE restock_product(
    p_product_id INT,
    p_quantity INT
)
LANGUAGE plpgsql
AS $$
BEGIN
    UPDATE products
    SET stock_quantity = stock_quantity + p_quantity
    WHERE product_id = p_product_id;

    COMMIT;
END;
$$;

CALL restock_product(205, 50);
```

9.6 Triggers

A **trigger** automatically runs a function in response to a table event (INSERT, UPDATE, DELETE).

BEFORE triggers can modify or reject the incoming row; **AFTER** triggers run once the change is committed to the table (typically used for logging or cascading effects); **INSTEAD OF** triggers apply to views. Row-level triggers fire once per affected row; statement-level triggers fire once per statement regardless of row count.

```
-- Automatically log every price change for auditing
CREATE TABLE price_change_log (
    product_id INT, old_price NUMERIC, new_price NUMERIC, changed_at TIMESTAMP DEFAULT NOW()
);

CREATE OR REPLACE FUNCTION log_price_change() RETURNS TRIGGER AS $$
BEGIN
    IF NEW.price <> OLD.price THEN
        INSERT INTO price_change_log(product_id, old_price, new_price)
        VALUES (OLD.product_id, OLD.price, NEW.price);
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_log_price_change
AFTER UPDATE ON products
FOR EACH ROW
EXECUTE FUNCTION log_price_change();
```

Practice Tasks

- 1) Write a CTE that computes each customer's total lifetime spend, then filter to customers above 500 using the CTE.
- 2) Write a recursive CTE that returns every descendant category of 'Electronics'.
- 3) Create a materialized view of daily revenue and explain when you would refresh it in a real pipeline.

Module 10: Performance Optimization

As NorthStar Commerce's tables grow from thousands to millions of rows, query design and indexing strategy determine whether a dashboard loads in zooms or 20 seconds. This module covers the tools data engineers use to keep queries fast.

10.1 Indexes

An **index** is an auxiliary data structure that lets the database find rows without scanning the entire table — much like a book's index lets you find a topic without reading every page.

Indexes speed up reads but add overhead to writes (every INSERT/UPDATE/DELETE must also update the index).

Index Type	Best For
B-tree (default)	Equality and range queries (=, <, >, BETWEEN, ORDER BY)
Hash	Pure equality lookups only, rarely chosen over B-tree in practice
GIN	Full-text search, JSONB, array containment
GiST	Geometric data, full-text search, range types

```
-- Speed up lookups of orders by customer
CREATE INDEX idx_orders_customer_id ON orders(customer_id);

-- Composite index: queries filtering on both columns together benefit most
CREATE INDEX idx_orders_customer_date ON orders(customer_id, order_date);

-- Partial index: only index the rows you actually query often
CREATE INDEX idx_orders_active ON orders(order_date) WHERE status <> 'cancelled';
```

When to avoid indexes: on small tables (a full scan is already fast), on columns rarely used in WHERE/JOIN/ORDER BY, and on tables with very heavy write volume where index maintenance cost outweighs the read benefit.

10.2 Partitions

Table partitioning splits one logical table into multiple physical pieces, so queries that target a narrow slice (e.g. one month) only scan the relevant partition rather than the whole table. Common strategies are range (e.g. by date), list (e.g. by country), and hash (evenly distributed by a hashed key).

```
-- Range-partition orders by month -- a very common pattern for time-series fact tables
CREATE TABLE orders_partitioned (
  order_id      INT,
  customer_id   INT,
  order_date    TIMESTAMP NOT NULL,
  status        VARCHAR(20),
  shipping_country VARCHAR(50)
) PARTITION BY RANGE (order_date);

CREATE TABLE orders_2026_01 PARTITION OF orders_partitioned
  FOR VALUES FROM ('2026-01-01') TO ('2026-02-01');

CREATE TABLE orders_2026_02 PARTITION OF orders_partitioned
  FOR VALUES FROM ('2026-02-01') TO ('2026-03-01');
```

The performance benefit is twofold: 'partition pruning' lets the planner skip irrelevant partitions entirely, and maintenance operations (like dropping old data) become near-instant by dropping a partition instead of running a slow DELETE.

10.3 Reading Query Plans: EXPLAIN and EXPLAIN ANALYZE

EXPLAIN shows the query planner's chosen execution plan without running the query;

EXPLAIN ANALYZE actually runs it and reports real timings alongside the plan — the single most important diagnostic tool for slow queries.

```
EXPLAIN ANALYZE
SELECT c.first_name, o.order_id, o.total_amount
FROM customers AS c
JOIN order_summary AS o ON o.customer_id = c.customer_id
WHERE o.order_date >= '2026-01-01';
```

Look for 'Seq Scan' (full table scan) on large tables where an 'Index Scan' would be expected — that is a strong signal a missing or unused index is hurting performance. Compare the planner's estimated row counts to the actual counts reported by ANALYZE; large discrepancies suggest stale table statistics.

10.4 Identifying and Fixing Slow Queries

- Select only the columns you need — **avoid SELECT *** in production queries and views.
- **Filter as early as possible**; push WHERE conditions down rather than filtering after a large join.
- **Index columns** used in JOIN conditions, WHERE clauses, and ORDER BY — especially foreign keys.

- **Avoid applying functions to indexed columns in WHERE** (e.g. WHERE DATE(order_date) = ... prevents index use); instead filter with a range: WHERE order_date >= '2026-01-01' AND order_date < '2026-01-02'.
- **Prefer EXISTS over IN** with large subqueries, and always be wary of NOT IN with nullable columns.
- **Use LIMIT** during development to avoid pulling huge result sets while iterating on a query.

10.5 Database-Specific Optimizations

PostgreSQL's query planner relies on up-to-date table statistics to choose good plans.

VACUUM reclaims storage from deleted/updated rows (PostgreSQL uses MVCC, so old row versions must be cleaned up); **ANALYZE** refreshes the planner's statistics about data distribution.

```
VACUUM ANALYZE orders;           -- reclaim space and refresh statistics for one table
VACUUM (ANALYZE, VERBOSE);       -- database-wide, with detailed output

-- autovacuum runs these automatically in the background on most PostgreSQL setups,
-- but data engineers still run them manually after very large bulk loads or deletes.
```

Practice Tasks

- 1) Run **EXPLAIN ANALYZE** on a query joining orders and order_items without any indexes, then again after adding an index on order_items.order_id — compare the plans.
- 2) Design a partitioning strategy for the payments table, and justify range vs. list vs. hash for this data.
- 3) Identify one query from Module 8 that would benefit from a composite index, and write the **CREATE INDEX** statement.

Module 11: AI & SQL

Large language models have changed how SQL gets written day to day, without changing what SQL is or why it matters. This module covers where AI genuinely helps, and where SQL fundamentals remain non-negotiable.

11.1 Generating SQL Queries with AI

Natural-language-to-SQL tools (built into modern BI platforms, IDE copilots, or chat assistants) can translate a plain-English question — 'show me revenue by category last quarter' — into a draft query against a described schema. This is fastest when the model is given the actual table and column names, as in the NorthStar Commerce schema at the front of this book, rather than left to guess.

```
-- A typical AI-generated first draft for:
-- "revenue by category for Q1 2026, highest first"
SELECT
    cat.category_name,
    SUM(oi.quantity * oi.unit_price) AS revenue
FROM order_items AS oi
JOIN products AS p ON p.product_id = oi.product_id
JOIN categories AS cat ON cat.category_id = p.category_id
JOIN orders AS o ON o.order_id = oi.order_id
WHERE o.order_date BETWEEN '2026-01-01' AND '2026-03-31'
GROUP BY cat.category_name
ORDER BY revenue DESC;
```

Treat AI-generated SQL as a first draft, not a final answer: verify it against the schema, check it handles NULLs and edge cases correctly, and confirm the logic (date boundaries, join direction, aggregation grain) matches what was actually asked — everything you have learned in Modules 1 through 10 is exactly what lets you evaluate that draft with confidence rather than trust it blindly.

11.2 AI-Assisted Query Optimization

AI tools can suggest indexes, rewrite a correlated subquery as a join, or flag an obviously inefficient pattern (like a function applied to an indexed column in WHERE) — but the EXPLAIN ANALYZE workflow from Module 10 is still how you validate whether a suggested change actually helps on your real data and data volumes.

11.3 Automating SQL Code Reviews

Some teams use AI to lint SQL pull requests for style consistency, missing WHERE clauses on UPDATE/DELETE, SELECT * in production views, or SQL-injection-prone string concatenation in application code — catching mechanical issues so human reviewers can focus on business logic.

11.4 Integrating SQL with AI/ML Pipelines

Cloud warehouses increasingly let you invoke machine learning directly from SQL — BigQuery ML and PostgreSQL's ML extensions (such as MADlib, or calling out to external model-serving endpoints) let you train or score models without leaving the SQL layer, which keeps feature engineering, training data assembly, and inference close to where the data already lives.

```
-- Conceptual example: BigQuery ML syntax for training a simple churn model
-- (shown for illustration; NorthStar's own warehouse may use a different tool)
CREATE OR REPLACE MODEL northstar.churn_model
OPTIONS(model_type = 'logistic_reg', input_label_cols = ['churned']) AS
SELECT
  customer_id,
  days_since_last_order,
  lifetime_orders,
  lifetime_value,
  churned
FROM northstar.customer_features;
```

11.5 Practical Use Cases

- **Predictive analytics** — SQL assembles the feature table (recency, frequency, monetary value per customer) that feeds a churn or lifetime-value model.
- **Anomaly detection** — window functions (Module 8) flag orders or revenue days that deviate sharply from a rolling average, a pattern often paired with an AI-based alerting layer.
- **Automated reporting** — scheduled SQL queries (via Airflow, dbt, or a warehouse's native scheduler) feed dashboards and AI-generated natural-language summaries of what changed.

The core lesson

AI is a powerful accelerant for writing and reviewing SQL, but it does not remove the need to understand joins, aggregation grain, NULL handling, and execution plans. The stronger your SQL fundamentals from Modules 1–10, the better you can direct, verify, and correct AI-generated SQL rather than being at its mercy.

Module 12: SQL Projects

These three capstone projects each combine most of the concepts from this book into one realistic, end-to-end piece of work on the NorthStar Commerce schema. Attempt each on your own before reading the discussion points.

Project 1: Data Warehousing (Data-Engineering Focus)

Goal: turn NorthStar's operational (OLTP) schema into an analytics-friendly star schema, then build the ETL/ELT logic to populate it.

1. Design the warehouse schema

A **star schema** centres one fact table (measurable events) around several dimension tables (descriptive context). For NorthStar, the natural fact table is order line items; dimensions describe the who, what, and when.

```
-- Fact table: one row per order line item, at the grain of an individual sale
CREATE TABLE fact_order_items (
  order_item_id INT PRIMARY KEY,
  order_id      INT,
  date_key      INT REFERENCES dim_date(date_key),
  customer_key  INT REFERENCES dim_customer(customer_key),
  product_key   INT REFERENCES dim_product(product_key),
  quantity      INT,
  unit_price    NUMERIC(10,2),
  revenue       NUMERIC(12,2)
);

CREATE TABLE dim_date (
  date_key INT PRIMARY KEY, full_date DATE, year INT, month INT, day INT,
  day_of_week VARCHAR(10)
);

CREATE TABLE dim_customer (
  customer_key INT PRIMARY KEY, customer_id INT, full_name VARCHAR(100), country
  VARCHAR(50), signup_date DATE
);

CREATE TABLE dim_product (
  product_key INT PRIMARY KEY, product_id INT, product_name VARCHAR(200),
  category_name VARCHAR(100), price NUMERIC(10,2)
);
```

2. Build the ELT pipeline (SQL)

```
-- Populate dim_date for 2025-2027 (a standard date-spine generator)
INSERT INTO dim_date (date_key, full_date, year, month, day, day_of_week)
SELECT
    TO_CHAR(d, 'YYYYMMDD')::INT,
    d,
    EXTRACT(YEAR FROM d),
    EXTRACT(MONTH FROM d),
    EXTRACT(DAY FROM d),
    TO_CHAR(d, 'Day')
FROM GENERATE_SERIES('2025-01-01'::DATE, '2027-12-31'::DATE, INTERVAL '1 day') AS d;

-- Populate the fact table by joining operational tables together
INSERT INTO fact_order_items (order_item_id, order_id, date_key, customer_key,
product_key, quantity, unit_price, revenue)
SELECT
    oi.order_item_id,
    oi.order_id,
    TO_CHAR(o.order_date, 'YYYYMMDD')::INT,
    dc.customer_key,
    dp.product_key,
    oi.quantity,
    oi.unit_price,
    oi.quantity * oi.unit_price
FROM order_items AS oi
JOIN orders AS o ON o.order_id = oi.order_id
JOIN dim_customer AS dc ON dc.customer_id = o.customer_id
JOIN dim_product AS dp ON dp.product_id = oi.product_id;
```

3. Optimize for analytical workloads

- **Partition fact_order_items by date_key range** (Module 10) since most warehouse queries filter by date.
- **Index foreign keys** (customer_key, product_key, date_key) to keep dimension joins fast.
- **Consider a materialized view** (Module 9) for the most common rollup — e.g. daily revenue by category — refreshed nightly.

Project 2: Exploratory Data Analysis (Analytics/Data-Science Focus)

Goal: understand customer purchasing behaviour through SQL-driven EDA, without writing a single line of Python – everything below runs directly against the operational schema.

1. Data cleaning and preprocessing

```
-- Check for obviously invalid data before analysing it
SELECT COUNT(*) FILTER (WHERE price <= 0) AS bad_prices,
       COUNT(*) FILTER (WHERE email IS NULL) AS missing_emails,
       COUNT(*) FILTER (WHERE quantity <= 0) AS bad_quantities
FROM products, customers, order_items; -- illustrative; run per table in practice
```

2. Statistical analysis: distributions and correlations

```
-- Distribution of order values in quartiles
SELECT
  NTILE(4) OVER (ORDER BY total_amount) AS quartile,
  MIN(total_amount), MAX(total_amount), AVG(total_amount)
FROM order_summary
GROUP BY 1
ORDER BY 1;

-- Simple correlation proxy: does higher review rating associate with repeat
purchases?
SELECT
  r.rating,
  COUNT(DISTINCT o2.order_id) AS follow_up_orders
FROM reviews AS r
JOIN orders AS o1 ON o1.customer_id = r.customer_id AND o1.order_date <=
r.review_date
LEFT JOIN orders AS o2 ON o2.customer_id = r.customer_id AND o2.order_date >
r.review_date
GROUP BY r.rating
ORDER BY r.rating;
```

3. Cohort analysis and time-series trends

```
-- Monthly signup cohorts, and how much each cohort had spent by month 3
WITH cohorts AS (
  SELECT customer_id, DATE_TRUNC('month', signup_date) AS cohort_month
  FROM customers
)
SELECT
  co.cohort_month,
  COUNT(DISTINCT co.customer_id) AS cohort_size,
  SUM(os.total_amount) FILTER (
    WHERE os.order_date < co.cohort_month + INTERVAL '3 months'
  ) AS revenue_within_3_months
FROM cohorts AS co
JOIN order_summary AS os ON os.customer_id = co.customer_id
GROUP BY co.cohort_month
ORDER BY co.cohort_month;
```

Project 3: Advanced Analytics

A/B testing analysis

Suppose half of NorthStar's customers saw a redesigned checkout page ('treatment') and half saw the original ('control'). SQL alone can compute the core comparison.

```
SELECT
    ab.variant,
    COUNT(DISTINCT o.order_id)::FLOAT / COUNT(DISTINCT ab.customer_id) AS
orders_per_customer,
    AVG(os.total_amount) AS avg_order_value
FROM ab_test_assignments AS ab
LEFT JOIN orders AS o ON o.customer_id = ab.customer_id AND o.order_date >=
ab.assigned_at
LEFT JOIN order_summary AS os ON os.order_id = o.order_id
GROUP BY ab.variant;
```

Churn prediction (feature table for SQL + ML)

```
-- Build a feature table: one row per customer, ready to feed a churn model
SELECT
    c.customer_id,
    CURRENT_DATE - MAX(os.order_date)::DATE AS days_since_last_order,
    COUNT(DISTINCT os.order_id) AS lifetime_orders,
    SUM(os.total_amount) AS lifetime_value,
    AVG(r.rating) AS avg_review_rating,
    CASE WHEN CURRENT_DATE - MAX(os.order_date)::DATE > 90 THEN 1 ELSE 0 END AS
churned
FROM customers AS c
LEFT JOIN order_summary AS os ON os.customer_id = c.customer_id
LEFT JOIN reviews AS r ON r.customer_id = c.customer_id
GROUP BY c.customer_id;
```

Real-time analytics with streaming data

In production, tools like Kafka or a warehouse's native streaming ingestion feed near-real-time data into staging tables; SQL then aggregates the last few minutes or hours using the same window and aggregate functions from Module 8 — the concepts don't change, only the freshness of the data being queried.

Wrapping up

If you have worked through all twelve modules and attempted these three projects against your own NorthStar Commerce database, you now have practical, end-to-end SQL experience spanning schema design, querying, performance tuning, and analytics — the same skill set used daily by data engineers, analysts, and data scientists in industry.

Next Steps by Role

If you are a...	Focus on
Beginner	Start with Module 1 and progress sequentially through Module 6.
Data Engineer	Modules 8 (Aggregation/Window Functions), 9 (Advanced Techniques), and 10 (Performance).
Data Analyst / Data Scientist	Modules 7 (Row-Level Functions), 8 (Aggregation), and Project 2 in Module 12 (EDA).
Any role	Module 11 (AI & SQL) for how modern tooling changes — and doesn't change — the job.

Additional Resources

Documentation

- PostgreSQL Documentation — <https://www.postgresql.org/docs/>
- SQL Style Guide — <https://www.sqlstyle.guide/>

Practice Platforms

- LeetCode (SQL Problems)
- HackerRank (SQL)
- StrataScratch
- Mode Analytics SQL Tutorial

Further Reading

- SQL for Data Analysis (O'Reilly)
- Learning PostgreSQL (Packt)
- SQL Cookbook, by Anthony Molinaro

This coursebook is modular by design — instructors and self-learners alike can reorder or skip modules to fit their role, goals, and prior experience, while still using the same NorthStar Commerce dataset as a consistent thread from first query to final capstone.